

beginning

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

data loading

```
df_encoded = pd.read_excel("/content/drive/MyDrive/capstone/datasets/5_encoded.xlsx")

df_mice_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/6_mice_context.xlsx")
df_mice_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/7_mice_no_context.xlsx")

df_knn_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/8_knn_context.xlsx")
df_knn_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/9_knn_no_context.xlsx")

df_rf_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/10_forest_context.xlsx")
df_rf_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/11_forest_no_context.xlsx")
```

```
def missing_data_report(df):
    # Calculate the percentage of missing data for each column
    missing_percentage = df.isna().mean() * 100

    # Get the data types of each column
    data_types = df.dtypes

    # Create a DataFrame for the missing data summary including data types
    missing_data_summary = pd.DataFrame({
        'Column': missing_percentage.index,
        'Missing Percentage': missing_percentage.values,
        'Data Type': data_types.values
    })

    # Define the four groups based on missing percentage
    group_1 = missing_data_summary[missing_data_summary['Missing Percentage'] > 70][['Column', 'Missing Percentage', 'Data Type']]
    group_2 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 40) & (missing_data_summary['Missing Percentage'] <= 70)][['Column', 'Missing Percentage', 'Data Type']]
```

```
group_3 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 20) & (missing_data_summary['Missing Percentage'] <= 40)][['Column', 'Missing Percentage']]
group_4 = missing_data_summary[missing_data_summary['Missing Percentage'] <= 20][['Column', 'Missing Percentage', 'Data Type']]

# Sort each group by missing percentage in descending order (most missing to least missing)
group_1 = group_1.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_2 = group_2.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_3 = group_3.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_4 = group_4.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)

# Rename columns for clarity
group_1.columns = ['Column', 'Missing %', 'Data Type']
group_2.columns = ['Column', 'Missing %', 'Data Type']
group_3.columns = ['Column', 'Missing %', 'Data Type']
group_4.columns = ['Column', 'Missing %', 'Data Type']

# Display each group one by one, from most missing to least missing
print("Group 1: Columns with >70% missing data")
print(group_1.to_string(index=False))
print("\nGroup 2: Columns with >40% to ≤70% missing data")
print(group_2.to_string(index=False))
print("\nGroup 3: Columns with >20% to ≤40% missing data")
print(group_3.to_string(index=False))
print("\nGroup 4: Columns with ≤20% missing data")
print(group_4.to_string(index=False))
```

df_encoded.tail()

	age	latitude	longitude	feels_like	temp_min	temp_max	air_pressure	air_humidity	wind_speed	wind_deg	...	gender_encoded_male	gender_encoded_third gender	workplace_enc
1612	15.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	0.0	0.0	
1613	27.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	0.0	0.0	
1614	9.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	...	1.0	0.0	
1615	NaN	23.898795	89.129554	NaN	12.5	23.7	1019.0	NaN	8.0	NaN	...	0.0	0.0	
1616	26.0	22.655478	89.794181	NaN	29.8	36.6	1003.8	NaN	3.8	NaN	...	0.0	0.0	

5 rows × 37 columns

missing_data_report(df_encoded)

```
Group 1: Columns with >70% missing data
Empty DataFrame
Columns: [Column, Missing %, Data Type]
Index: []

Group 2: Columns with >40% to ≤70% missing data
      Column  Missing %  Data Type
feels_like  53.061224   float64
air_humidity 53.061224   float64
wind_deg     53.061224   float64
clouds_sky   53.061224   float64
weather_main_encoded 52.999382   float64
```

```
weather_description_encoded 52.999382 float64
financial_condition_encoded 45.516388 float64
```

```
Group 3: Columns with >20% to ≤40% missing data
      Column  Missing %  Data Type
workplace_encoded_semi-urban 37.909709 float64
workplace_encoded_urban     37.909709 float64
workplace_encoded_rural     37.909709 float64
      air_pressure 30.426716 float64
      wind_speed  30.179344 float64
      temp_min    29.870130 float64
      temp_max    29.560915 float64
marital_status_encoded      23.933210 float64
```

```
Group 4: Columns with ≤20% missing data
      Column  Missing %  Data Type
profession_group_encoded 18.923933 float64
reason_encoded           16.821274 float64
      age        12.739641 float64
      time_encoded 12.059369 float64
age_group_encoded        7.421150 float64
      latitude   6.493506 float64
      longitude  6.493506 float64
method_encoded           2.906617 float64
suicide_month            2.411874 float64
suicide_day              2.411874 float64
suicide_weekday_encoded  2.411874 float64
suicide_week             2.411874 float64
suicide_year             2.411874 float64
religion_encoded_buddhist 2.411874 float64
religion_encoded_muslim   2.411874 float64
religion_encoded_indigenous 2.411874 float64
religion_encoded_hindu    2.411874 float64
religion_encoded_christian 2.411874 float64
hometown_encoded         1.669759 float64
gender_encoded_third gender 0.432900 float64
gender_encoded_female     0.432900 float64
gender_encoded_male       0.432900 float64
```

```
import pandas as pd

def create_datasets(df_encoded):
    # Columns with >40% to ≤70% missing (Group 2)
    group2_columns = [
        'feels_like', 'air_humidity', 'wind_deg', 'clouds_sky',
        'weather_main_encoded', 'weather_description_encoded', 'financial_condition_encoded'
    ]

    # Columns with >20% to ≤40% missing (Group 3)
    group3_columns = [
        'workplace_encoded_semi-urban', 'workplace_encoded_urban', 'workplace_encoded_rural',
        'air_pressure', 'wind_speed', 'temp_min', 'temp_max', 'marital_status_encoded'
    ]

    # Columns with ≤20% missing (Group 4)
    group4_columns = [
        'profession_group_encoded', 'reason_encoded', 'age', 'time_encoded',
        'age_group_encoded', 'latitude', 'longitude', 'method_encoded',
        'suicide_month', 'suicide_day', 'suicide_weekday_encoded', 'suicide_week',

```

```

'suicide_year', 'religion_encoded_buddhist', 'religion_encoded_muslim',
'religion_encoded_indigenous', 'religion_encoded_hindu', 'religion_encoded_christian',
'hometown_encoded', 'gender_encoded_third gender', 'gender_encoded_female',
'gender_encoded_male'
]

```

```

# Dataset 1: Columns with <=20% missing
df1_columns = group4_columns
df1 = df_encoded[df1_columns].copy().dropna()

```

```

# Dataset 2: Columns with <=40% missing (Group 3 + Group 4)
df2_columns = group3_columns + group4_columns
df2 = df_encoded[df2_columns].copy().dropna()

```

```

# Dataset 3: All columns (Group 2 + Group 3 + Group 4)
df3_columns = group2_columns + group3_columns + group4_columns
df3 = df_encoded[df3_columns].copy().dropna()

```

```

return df1, df2, df3

```

```

df_encoded_20, df_encoded_40, df_encoded_70 = create_datasets(df_encoded)

```

```

display(df_encoded_20.shape)
display(df_encoded_40.shape)
display(df_encoded_70.shape)

```

```

(853, 22)
(403, 30)
(57, 37)

```

✖ Leaving the df_encoded_70 dataframe since very low number of rows

```

df_mice_context = df_mice_context.dropna()
df_mice_no_context = df_mice_no_context.dropna()
df_knn_context = df_knn_context.dropna()
df_knn_no_context = df_knn_no_context.dropna()
df_rf_context = df_rf_context.dropna()
df_rf_no_context = df_rf_no_context.dropna()

```

```

display(df_mice_context.shape)
display(df_mice_no_context.shape)
display(df_knn_context.shape)
display(df_knn_no_context.shape)
display(df_rf_context.shape)
display(df_rf_no_context.shape)

```

```

(1613, 37)
(1613, 37)
(1617, 37)
(1617, 37)
(1617, 29)
(1617, 29)

```

```
dataframes = [df_encoded_20, df_encoded_40, df_mice_context, df_mice_no_context, df_knn_context, df_knn_no_context, df_rf_context, df_rf_no_context]
df_names = ["df_encoded_20", "df_encoded_40", "df_mice_context", "df_mice_no_context", "df_knn_context", "df_knn_no_context", "df_rf_context", "df_rf_no_context"]
```

```
# from sklearn.preprocessing import StandardScaler
# from sklearn.decomposition import PCA
# from sklearn.cluster import KMeans
# from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score
# from tqdm import tqdm
# import pandas as pd
# import numpy as np

# # -----
# # Stage 0: Scaling + PCA
# # -----
# def preprocess_data(df, use_pca=True, pca_variance=0.99, random_state=42):
#     X = df.copy()
#     scaler = StandardScaler()
#     X_scaled = scaler.fit_transform(X)

#     if use_pca:
#         pca = PCA(n_components=pca_variance, random_state=random_state)
#         X_scaled = pca.fit_transform(X_scaled)

#     return X_scaled

# # -----
# # Stage 1: KMeans grid search
# # -----
# def kmeans_grid_search(
#     X_scaled,
#     n_clusters_range=range(2, 21),
#     init_methods=['k-means++', 'random'],
#     n_init_options=[10, 20, 50],
#     max_iter_options=[300, 500],
#     feedback=True,
#     random_state=42
# ):
#     best_score = -np.inf
#     best_params = {}
#     best_labels = None
#     metrics_records = []

#     total_iterations = len(n_clusters_range) * len(init_methods) * len(n_init_options) * len(max_iter_options)
#     iteration = 0

#     for n_clusters in tqdm(n_clusters_range, desc="KMeans Hyperparameter Search"):
#         for init in init_methods:
#             for n_init in n_init_options:
#                 for max_iter in max_iter_options:
#                     iteration += 1
#                     try:
#                         kmeans = KMeans(
#                             n_clusters=n_clusters,
#                             init=init,
#                             n_init=n_init,
```

```

#             max_iter=max_iter,
#             random_state=random_state
#         )
#         labels = kmeans.fit_predict(X_scaled)

#         # Metrics
#         sil = silhouette_score(X_scaled, labels)
#         ch = calinski_harabasz_score(X_scaled, labels)
#         db = davies_bouldin_score(X_scaled, labels)
#         combined_score = sil / (db + 1e-12)

#         metrics_records.append({
#             'n_clusters': n_clusters,
#             'init': init,
#             'n_init': n_init,
#             'max_iter': max_iter,
#             'silhouette': sil,
#             'calinski_harabasz': ch,
#             'davies_bouldin': db,
#             'combined_score': combined_score
#         })

#         if combined_score > best_score:
#             best_score = combined_score
#             best_params = {
#                 'n_clusters': n_clusters,
#                 'init': init,
#                 'n_init': n_init,
#                 'max_iter': max_iter
#             }
#             best_labels = labels

#     except Exception as e:
#         if feedback:
#             print(f"⚠ Skipped config due to error: {e}")

#     metrics_df = pd.DataFrame(metrics_records)
#     return best_labels, best_params, metrics_df, best_score

# # -----
# # Stage 2: Fine-tuning around best n_clusters
# # -----
# def kmeans_fine_tune(X_scaled, best_labels, best_params, best_score, random_state=42):
#     fine_tune_range = range(max(2, best_params['n_clusters'] - 2), best_params['n_clusters'] + 3)
#     for n_clusters in fine_tune_range:
#         try:
#             kmeans = KMeans(
#                 n_clusters=n_clusters,
#                 init=best_params['init'],
#                 n_init=best_params['n_init'],
#                 max_iter=best_params['max_iter'],
#                 random_state=random_state
#             )
#             labels = kmeans.fit_predict(X_scaled)
#             sil = silhouette_score(X_scaled, labels)
#             db = davies_bouldin_score(X_scaled, labels)

```

```

#         combined_score = sil / (db + 1e-12)

#         if combined_score > best_score:
#             best_score = combined_score
#             best_params['n_clusters'] = n_clusters
#             best_labels = labels
#     except:
#         continue
#     return best_labels, best_params, best_score

# # -----
# # Stage 3: Full KMeans pipeline
# # -----
# def kmeans_pipeline_modular(
#     df,
#     n_clusters_range=range(2, 21),
#     init_methods=['k-means++', 'random'],
#     n_init_options=[10, 20, 50],
#     max_iter_options=[300, 500],
#     use_pca=True,
#     pca_variance=0.99,
#     feedback=True,
#     random_state=42
# ):
#     # Stage 0: Preprocess
#     X_scaled = preprocess_data(df, use_pca=use_pca, pca_variance=pca_variance, random_state=random_state)

#     # Stage 1: Grid search
#     best_labels, best_params, metrics_df, best_score = kmeans_grid_search(
#         X_scaled,
#         n_clusters_range=n_clusters_range,
#         init_methods=init_methods,
#         n_init_options=n_init_options,
#         max_iter_options=max_iter_options,
#         feedback=feedback,
#         random_state=random_state
#     )

#     # Stage 2: Fine-tune around best n_clusters
#     best_labels, best_params, best_score = kmeans_fine_tune(
#         X_scaled, best_labels, best_params, best_score, random_state=random_state
#     )

#     # Stage 3: Attach cluster column
#     clustered_df = df.copy()
#     clustered_df['kmeans_cluster'] = best_labels

#     if feedback:
#         print(f"\n👍 Best KMeans params: {best_params}")
#         print(f"Silhouette score of best clustering: {silhouette_score(X_scaled, best_labels):.4f}")
#         print(f"Calinski-Harabasz score: {calinski_harabasz_score(X_scaled, best_labels):.4f}")
#         print(f"Davies-Bouldin score: {davies_bouldin_score(X_scaled, best_labels):.4f}")

#     return clustered_df, best_params, metrics_df

```

```

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score
from joblib import Parallel, delayed
import pandas as pd
import numpy as np
from tqdm import tqdm

# -----
# Stage 0: Scaling + PCA
# -----
def preprocess_data(df, use_pca=True, pca_variance=0.99, random_state=42):
    X = df.copy()
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    if use_pca:
        pca = PCA(n_components=pca_variance, random_state=random_state)
        X_scaled = pca.fit_transform(X_scaled)

    return X_scaled

# -----
# Stage 1: Single KMeans run
# -----
def run_single_kmeans(X_scaled, n_clusters, init, n_init, max_iter, random_state):
    try:
        kmeans = KMeans(
            n_clusters=n_clusters,
            init=init,
            n_init=n_init,
            max_iter=max_iter,
            random_state=random_state,
        )
        labels = kmeans.fit_predict(X_scaled)
        sil = silhouette_score(X_scaled, labels)
        ch = calinski_harabasz_score(X_scaled, labels)
        db = davies_bouldin_score(X_scaled, labels)
        combined_score = sil / (db + 1e-12)
        return {
            'n_clusters': n_clusters,
            'init': init,
            'n_init': n_init,
            'max_iter': max_iter,
            'silhouette': sil,
            'calinski_harabasz': ch,
            'davies_bouldin': db,
            'combined_score': combined_score,
            'labels': labels
        }
    except Exception as e:
        return None

# -----
# Stage 1: KMeans grid search (parallel)

```

```

# -----
def kmeans_grid_search(
    X_scaled,
    n_clusters_range=range(2, 21),
    init_methods=['k-means++', 'random'],
    n_init_options=[10, 20, 50],
    max_iter_options=[300, 500],
    feedback=True,
    random_state=42
):
    # Build all parameter combinations
    param_list = [
        (n_clusters, init, n_init, max_iter)
        for n_clusters in n_clusters_range
        for init in init_methods
        for n_init in n_init_options
        for max_iter in max_iter_options
    ]

    # Parallel execution
    results = Parallel(n_jobs=-1, verbose=5)(
        delayed(run_single_kmeans)(X_scaled, *params, random_state) for params in param_list
    )

    # Filter out failed runs
    results = [r for r in results if r is not None]

    # Find best
    best_run = max(results, key=lambda x: x['combined_score'])
    best_labels = best_run['labels']
    best_params = {
        'n_clusters': best_run['n_clusters'],
        'init': best_run['init'],
        'n_init': best_run['n_init'],
        'max_iter': best_run['max_iter']
    }
    metrics_df = pd.DataFrame([r for r in results if r is not None]).drop(columns=['labels'])
    best_score = best_run['combined_score']

    return best_labels, best_params, metrics_df, best_score

# -----
# Stage 2: Fine-tuning around best n_clusters
# -----
def kmeans_fine_tune(X_scaled, best_labels, best_params, best_score, random_state=42):
    fine_tune_range = range(max(2, best_params['n_clusters'] - 2), best_params['n_clusters'] + 3)
    for n_clusters in fine_tune_range:
        try:
            kmeans = KMeans(
                n_clusters=n_clusters,
                init=best_params['init'],
                n_init=best_params['n_init'],
                max_iter=best_params['max_iter'],
                random_state=random_state
            )
            labels = kmeans.fit_predict(X_scaled)

```

```

        sil = silhouette_score(X_scaled, labels)
        db = davies_bouldin_score(X_scaled, labels)
        combined_score = sil / (db + 1e-12)

        if combined_score > best_score:
            best_score = combined_score
            best_params['n_clusters'] = n_clusters
            best_labels = labels
    except:
        continue
    return best_labels, best_params, best_score

# -----
# Stage 3: Full KMeans pipeline
# -----
def kmeans_pipeline(
    df,
    n_clusters_range=range(2, 21),
    init_methods=['k-means++', 'random'],
    n_init_options=[10, 20, 50],
    max_iter_options=[300, 500],
    use_pca=True,
    pca_variance=0.99,
    feedback=True,
    random_state=42
):
    # Stage 0: Preprocess
    X_scaled = preprocess_data(df, use_pca=use_pca, pca_variance=pca_variance, random_state=random_state)

    # Stage 1: Parallel Grid search
    best_labels, best_params, metrics_df, best_score = kmeans_grid_search(
        X_scaled,
        n_clusters_range=n_clusters_range,
        init_methods=init_methods,
        n_init_options=n_init_options,
        max_iter_options=max_iter_options,
        feedback=feedback,
        random_state=random_state
    )

    # Stage 2: Fine-tune around best n_clusters
    best_labels, best_params, best_score = kmeans_fine_tune(
        X_scaled, best_labels, best_params, best_score, random_state=random_state
    )

    # Stage 3: Attach cluster column
    clustered_df = df.copy()
    clustered_df['kmeans_cluster'] = best_labels

    if feedback:
        print(f"\n✅ Best KMeans params: {best_params}")
        print(f"Silhouette score: {silhouette_score(X_scaled, best_labels):.4f}")
        print(f"Calinski-Harabasz score: {calinski_harabasz_score(X_scaled, best_labels):.4f}")
        print(f"Davies-Bouldin score: {davies_bouldin_score(X_scaled, best_labels):.4f}")

```

```
return clustered_df, best_params, metrics_df
```

```
# -----
# Stage 4: Run pipeline on multiple dataframes
# -----
for i, (df, name) in enumerate(zip(dataframes, df_names)):
    print(f"\nRunning KMeans pipeline on {name} ...")

    clustered_df, best_params, metrics_df = kmeans_pipeline(
        df,
        n_clusters_range=range(2, 21),
        init_methods=['k-means++', 'random'],
        n_init_options=[10, 20, 50],
        max_iter_options=[300, 500],
        use_pca=True,
        pca_variance=0.99,
        feedback=True,
        random_state=42
    )

    dataframes[i]['kmeans_cluster'] = clustered_df['kmeans_cluster']
    print(f"Best params for {name}: {best_params}")
    print(f"Clustered column 'kmeans_cluster' added to {name}.")
```

```
Running KMeans pipeline on df_encoded_20 ...
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
[Parallel(n_jobs=-1)]: Done 14 tasks | elapsed: 2.7s
[Parallel(n_jobs=-1)]: Done 68 tasks | elapsed: 8.5s
[Parallel(n_jobs=-1)]: Done 158 tasks | elapsed: 18.9s
[Parallel(n_jobs=-1)]: Done 228 out of 228 | elapsed: 30.0s finished
```

```
✓ Best KMeans params: {'n_clusters': 10, 'init': 'k-means++', 'n_init': 50, 'max_iter': 300}
Silhouette score: 0.1192
Calinski-Harabasz score: 82.1256
Davies-Bouldin score: 1.4118
Best params for df_encoded_20: {'n_clusters': 10, 'init': 'k-means++', 'n_init': 50, 'max_iter': 300}
Clustered column 'kmeans_cluster' added to df_encoded_20.
```

```
Running KMeans pipeline on df_encoded_40 ...
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
[Parallel(n_jobs=-1)]: Done 36 tasks | elapsed: 1.5s
[Parallel(n_jobs=-1)]: Done 228 out of 228 | elapsed: 16.1s finished
```

```
✓ Best KMeans params: {'n_clusters': 8, 'init': 'k-means++', 'n_init': 50, 'max_iter': 300}
Silhouette score: 0.1297
Calinski-Harabasz score: 30.7495
Davies-Bouldin score: 1.7444
Best params for df_encoded_40: {'n_clusters': 8, 'init': 'k-means++', 'n_init': 50, 'max_iter': 300}
Clustered column 'kmeans_cluster' added to df_encoded_40.
```

```
Running KMeans pipeline on df_mice_context ...
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
[Parallel(n_jobs=-1)]: Done 24 tasks | elapsed: 2.6s
[Parallel(n_jobs=-1)]: Done 128 tasks | elapsed: 21.9s
[Parallel(n_jobs=-1)]: Done 218 tasks | elapsed: 42.0s
```

```
[Parallel(n_jobs=-1)]: Done 228 out of 228 | elapsed: 44.4s finished
```

✓ Best KMeans params: {'n_clusters': 2, 'init': 'random', 'n_init': 20, 'max_iter': 300}
 Silhouette score: 0.1916
 Calinski-Harabasz score: 374.2899
 Davies-Bouldin score: 1.9572
 Best params for df_mice_context: {'n_clusters': 2, 'init': 'random', 'n_init': 20, 'max_iter': 300}
 Clustered column 'kmeans_cluster' added to df_mice_context.

Running KMeans pipeline on df_mice_no_context ...
 [Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
 [Parallel(n_jobs=-1)]: Done 24 tasks | elapsed: 2.9s
 [Parallel(n_jobs=-1)]: Done 130 tasks | elapsed: 20.5s
 [Parallel(n_jobs=-1)]: Done 220 tasks | elapsed: 41.0s
 [Parallel(n_jobs=-1)]: Done 228 out of 228 | elapsed: 43.1s finished

✓ Best KMeans params: {'n_clusters': 2, 'init': 'k-means++', 'n_init': 10, 'max_iter': 300}
 Silhouette score: 0.2130
 Calinski-Harabasz score: 420.2920
 Davies-Bouldin score: 1.8389
 Best params for df_mice_no_context: {'n_clusters': 2, 'init': 'k-means++', 'n_init': 10, 'max_iter': 300}
 Clustered column 'kmeans_cluster' added to df_mice_no_context.

Running KMeans pipeline on df_knn_context ...
 [Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.

```
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score
from scipy.spatial.distance import cdist
import numpy as np

def evaluate_clusters(
    df,
    label_col,
    ignore_label=None,      # e.g., -1 for DBSCAN, None for KMeans/Agglo/KNM
    sil_metric="euclidean", # match your clustering distance
    sample_size=None,
    random_state=42,
    return_dict=False
):
    """
    General-purpose clustering evaluation function.
    Works for Agglomerative, DBSCAN, KMeans, Spectral, GMM, etc.
    """

    rng = np.random.RandomState(random_state)

    X = df.drop(columns=[label_col]).values
    labels = df[label_col].values

    # Optionally ignore a label (e.g., DBSCAN noise = -1)
    if ignore_label is not None:
        mask = labels != ignore_label
        X, labels = X[mask], labels[mask]

    # Optional subsample for speed (applies to all metrics)
    if sample_size is not None and X.shape[0] > sample_size:
        idx = rng.choice(X.shape[0], size=sample_size, replace=False)
        X, labels = X[idx], labels[idx]
```

```

unique_clusters = np.unique(labels)
n_clusters = len(unique_clusters)

if n_clusters <= 1:
    print("⚠ Only one cluster found. Metrics cannot be computed robustly.")
    return {} if return_dict else None

results = {}

# Core indices
try:
    results["Silhouette Score"] = {
        "score": silhouette_score(X, labels, metric=sil_metric),
        "range": "[-1, 1]",
        "better": "higher"
    }
except Exception:
    results["Silhouette Score"] = {"score": np.nan, "range": "[-1, 1]", "better": "higher"}

try:
    results["Calinski-Harabasz Index"] = {
        "score": calinski_harabasz_score(X, labels),
        "range": "[0, ∞)",
        "better": "higher"
    }
except Exception:
    results["Calinski-Harabasz Index"] = {"score": np.nan, "range": "[0, ∞)", "better": "higher"}

try:
    results["Davies-Bouldin Index"] = {
        "score": davies_bouldin_score(X, labels),
        "range": "[0, ∞)",
        "better": "lower"
    }
except Exception:
    results["Davies-Bouldin Index"] = {"score": np.nan, "range": "[0, ∞)", "better": "lower"}

# Dunn Index (guard against singletons / zero intra-distances)
def dunn_index(X, labels):
    eps = 1e-12
    uniq = np.unique(labels)
    # Intra-cluster diameters
    intra = []
    for k in uniq:
        pts = X[labels == k]
        if pts.shape[0] >= 2:
            d = cdist(pts, pts)
            intra.append(np.max(d))
        else:
            intra.append(0.0) # singleton cluster
    denom = max(max(intra), eps)

    # Inter-cluster minimum distance
    inter_min = np.inf
    for i in range(len(uniq)):

```

```

        for j in range(i + 1, len(uniq)):
            Xi = X[labels == uniq[i]]
            Xj = X[labels == uniq[j]]
            d = cdist(Xi, Xj)
            inter_min = min(inter_min, np.min(d))
        if not np.isfinite(inter_min):
            return np.nan
        return inter_min / denom

# Xie-Beni Index (guard against identical centroids)
def xie_beni_index(X, labels):
    eps = 1e-12
    uniq = np.unique(labels)
    centroids = np.array([X[labels == k].mean(axis=0) for k in uniq])
    # Compactness
    compact = 0.0
    for i, k in enumerate(uniq):
        pts = X[labels == k]
        diff = pts - centroids[i]
        compact += np.sum(diff * diff)
    # Min centroid separation
    min_csep = np.inf
    for i in range(len(centroids)):
        for j in range(i + 1, len(centroids)):
            d = np.sum((centroids[i] - centroids[j]) ** 2)
            if d < min_csep:
                min_csep = d
    if not np.isfinite(min_csep) or min_csep <= eps:
        return np.nan
    return compact / (X.shape[0] * min_csep)

try:
    results["Dunn Index"] = {
        "score": dunn_index(X, labels),
        "range": "[0, ∞)",
        "better": "higher"
    }
except Exception:
    results["Dunn Index"] = {"score": np.nan, "range": "[0, ∞)", "better": "higher"}

try:
    results["Xie-Beni Index"] = {
        "score": xie_beni_index(X, labels),
        "range": "[0, ∞)",
        "better": "lower"
    }
except Exception:
    results["Xie-Beni Index"] = {"score": np.nan, "range": "[0, ∞)", "better": "lower"}

# Pretty print
print("\n📊 Clustering Evaluation Results:")
for metric, info in results.items():
    score = info['score']
    score_str = f"{score:.4f}" if isinstance(score, (int, float)) and np.isfinite(score) else "NaN"
    print(f"{metric}: {score_str} ----- Range: {info['range']} ----- {info['better'].capitalize()} is better")

```

```
return results if return_dict else None
```

```
for df, name in zip(dataframes, df_names):
    cluster_col = 'kmeans_cluster'
    n_clusters = df[cluster_col].nunique() if cluster_col in df.columns else "Not assigned yet"

    print("\n-----")
    print(f"Evaluation metrics for {name}: total clusters: {n_clusters}")
    print("=====")

    # Evaluate on original dataframe with cluster labels
    evaluate_clusters(df, cluster_col, sil_metric='euclidean')
```

```
-----
Evaluation metrics for df_encoded_20: total clusters: 10
=====
```

Clustering Evaluation Results:

```
Silhouette Score: -0.2127 ----- Range: [-1, 1] ----- Higher is better
Calinski-Harabasz Index: 28.0411 ----- Range: [0, ∞) ----- Higher is better
Davies-Bouldin Index: 4.2392 ----- Range: [0, ∞) ----- Lower is better
Dunn Index: 0.0206 ----- Range: [0, ∞) ----- Higher is better
Xie-Beni Index: 11.6062 ----- Range: [0, ∞) ----- Lower is better
```

```
-----
Evaluation metrics for df_encoded_40: total clusters: 8
=====
```

Clustering Evaluation Results:

```
Silhouette Score: -0.0719 ----- Range: [-1, 1] ----- Higher is better
Calinski-Harabasz Index: 1281.8871 ----- Range: [0, ∞) ----- Higher is better
Davies-Bouldin Index: 4.9323 ----- Range: [0, ∞) ----- Lower is better
Dunn Index: 0.0085 ----- Range: [0, ∞) ----- Higher is better
Xie-Beni Index: 29.7254 ----- Range: [0, ∞) ----- Lower is better
```

```
-----
Evaluation metrics for df_mice_context: total clusters: 2
=====
```

Clustering Evaluation Results:

```
Silhouette Score: 0.7088 ----- Range: [-1, 1] ----- Higher is better
Calinski-Harabasz Index: 7649.1781 ----- Range: [0, ∞) ----- Higher is better
Davies-Bouldin Index: 0.3915 ----- Range: [0, ∞) ----- Lower is better
Dunn Index: 0.0110 ----- Range: [0, ∞) ----- Higher is better
Xie-Beni Index: 0.0520 ----- Range: [0, ∞) ----- Lower is better
```

```
-----
Evaluation metrics for df_mice_no_context: total clusters: 2
=====
```

Clustering Evaluation Results:

```
Silhouette Score: 0.7921 ----- Range: [-1, 1] ----- Higher is better
Calinski-Harabasz Index: 13154.9105 ----- Range: [0, ∞) ----- Higher is better
Davies-Bouldin Index: 0.2627 ----- Range: [0, ∞) ----- Lower is better
Dunn Index: 0.0308 ----- Range: [0, ∞) ----- Higher is better
Xie-Beni Index: 0.0303 ----- Range: [0, ∞) ----- Lower is better
```

```
-----
Evaluation metrics for df_knn_context: total clusters: 2
=====
```

Clustering Evaluation Results:

```
Silhouette Score: 0.7849 ----- Range: [-1, 1] ----- Higher is better
Calinski-Harabasz Index: 8708.8960 ----- Range: [0, ∞) ----- Higher is better
Davies-Bouldin Index: 0.2728 ----- Range: [0, ∞) ----- Lower is better
Dunn Index: 0.0325 ----- Range: [0, ∞) ----- Higher is better
Xie-Beni Index: 0.0363 ----- Range: [0, ∞) ----- Lower is better
-----
```

```
# from google.colab import files
# import pandas as pd

# # Assuming dataframes and df_names are already defined and populated
# # from previous cells.

# output_dir = "/content/kmeans_exported_dfs"
# os.makedirs(output_dir, exist_ok=True)

# exported_files = []

# for df, name in zip(dataframes, df_names):
#     # Construct the output filename
#     output_filename = f"kmeans_{name}.xlsx"
#     output_filepath = os.path.join(output_dir, output_filename)

#     # Export the dataframe to an Excel file
#     try:
#         df.to_excel(output_filepath, index=False)
#         print(f"Exported {name} to {output_filepath}")
#         exported_files.append(output_filepath)
#     except Exception as e:
#         print(f"Error exporting {name}: {e}")

# # Provide a way to download the files
# if exported_files:
#     print("\nYour files are ready for download:")
#     for f in exported_files:
#         print(f)
#     # You can manually download from the file browser in Colab,
#     # or use the following code to download them programmatically:
#     for f in exported_files:
#         files.download(f)
# else:
#     print("\nNo files were exported.")
```

```
Exported df_encoded_20 to /content/kmeans_exported_dfs/kmeans_df_encoded_20.xlsx
Exported df_encoded_40 to /content/kmeans_exported_dfs/kmeans_df_encoded_40.xlsx
Exported df_mice_context to /content/kmeans_exported_dfs/kmeans_df_mice_context.xlsx
Exported df_mice_no_context to /content/kmeans_exported_dfs/kmeans_df_mice_no_context.xlsx
Exported df_knn_context to /content/kmeans_exported_dfs/kmeans_df_knn_context.xlsx
Exported df_knn_no_context to /content/kmeans_exported_dfs/kmeans_df_knn_no_context.xlsx
Exported df_rf_context to /content/kmeans_exported_dfs/kmeans_df_rf_context.xlsx
Exported df_rf_no_context to /content/kmeans_exported_dfs/kmeans_df_rf_no_context.xlsx
```

Your files are ready for download:

```
/content/kmeans_exported_dfs/kmeans_df_encoded_20.xlsx  
/content/kmeans_exported_dfs/kmeans_df_encoded_40.xlsx  
/content/kmeans_exported_dfs/kmeans_df_mice_context.xlsx  
/content/kmeans_exported_dfs/kmeans_df_mice_no_context.xlsx  
/content/kmeans_exported_dfs/kmeans_df_knn_context.xlsx  
/content/kmeans_exported_dfs/kmeans_df_knn_no_context.xlsx  
/content/kmeans_exported_dfs/kmeans_df_rf_context.xlsx  
/content/kmeans_exported_dfs/kmeans_df_rf_no_context.xlsx
```